

**LAPORAN TUGAS AKHIR EAS
PEMROGAMAN BERORIENTASI OBJEK**



Disusun Oleh:

Arva Aprilia Salsabilla	1482500053
Mochammad Afrizal Wahyu I.	1482500067
Giska Erdian Claricha	1482500068

Dosen Pengampu:

Bagus Hardiansyah, S.Kom., M.Si.

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

JUNI 2026

DAFTAR ISI

BAB I.....	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah.....	1
1.3 Tujuan dan Manfaat	1
BAB II.....	2
LANDASAN TEORI.....	2
2.1 Class dan Object	2
2.2 Inheritance.....	2
2.3 Bahasa Pemrograman Python	2
2.4 Encapsulation.....	2
BAB III	3
ANALISIS DAN PERANCANGAN	3
3.1 Deskripsi Sistem	3
3.2 Identifikasi Class.....	3
3.3 Struktur Class.....	4
3.4 Data Produk yang Tersedia	4
3.5 Diagram Class	5
3.6 Flowchart Alur Program	7
3.7 Alur Program	7
BAB IV	8
IMPLEMENTASI DAN PENGUJIAN	8
4.1 Konsep OOP yang Diterapkan	8
4.2 Class Produk (Parent Class).....	8
4.3 Class Makanan (Inheritance dari Produk).....	9
4.4 Class Minuman (Inheritance dari Produk).....	9
4.5 Class DetailBelanja.....	9
4.6 Class Transaksi.....	10
4.7 Pembuatan Object dan Eksekusi Program	11
4.8 Menyimpan Seluruh Produk ke dalam List.....	11
4.9 Menu Pembelian (Perulangan while)	12
5.0 Proses Pembayaran dan Pembuatan Object Transaksi	13
5.1 Hasil Output Program	14

5.1	Tabel Hasil Pengujian	15
BAB V		16
KESIMPULAN		16
DAFTAR PUSTAKA		17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era digitalisasi saat ini, efisiensi dan akurasi dalam pengelolaan transaksi penjualan menjadi kunci utama keberhasilan suatu usaha. Banyak pelaku usaha yang masih menggunakan metode pencatatan manual atau konvensional dalam menghitung total belanjaan, mengelola stok barang, dan mencetak bukti transaksi. Metode manual ini memiliki beberapa kelemahan yang cukup signifikan, Kesalahan dalam perhitungan harga total atau kembalian uang yang dapat merugikan penjual maupun pembeli, Antrean pelanggan menjadi lebih panjang karena proses pencatatan barang dan perhitungan memakan waktu lama. Dalam sistem kasir, terdapat produk dengan karakteristik umum seperti nama, harga, dan stok, namun juga terdapat produk dengan karakteristik khusus, misalnya makanan yang memiliki tanggal kedaluwarsa dan minuman yang memiliki ukuran kemasan. Oleh karena itu disini kita akan membuat Aplikasi Kasir Sederhana dengan Konsep Class, Object, dan Inheritance, aplikasi ini dirancang untuk mengotomatisasi proses perhitungan transaksi secara cepat, tepat, dan akurat.

1.2 Rumusan Masalah

- 1.2.1 Bagaimana menerapkan konsep Class dan Object pada sistem aplikasi kasir sederhana?
- 1.2.2 Bagaimana menerapkan konsep Inheritance pada Class Produk sehingga dapat diturunkan menjadi Class Makanan dan Minuman?

1.3 Tujuan dan Manfaat

- 1.3.1 Menerapkan konsep Class dan Object dalam membangun program aplikasi kasir sederhana.
- 1.3.2 Melatih kemampuan merancang struktur program yang modular dan mudah dikembangkan menggunakan paradigma OOP.
- 1.3.3 Membantu memahami secara konkret perbedaan antara Class sebagai rancangan dan Object sebagai hasil nyata dari rancangan tersebut.

BAB II

LANDASAN TEORI

2.1 Class dan Object

Class adalah konsep dalam OOP yang digunakan untuk membuat struktur dasar untuk membuat objek. Di dalam class terdapat atribut atau properti yang merepresentasikan karakteristik dari objek, serta metode atau fungsi yang merepresentasikan tindakan atau perilaku yang dapat dilakukan oleh objek.

Object atau objek adalah hasil dari deklarasi atau instance sebuah class. Dalam objek, kita dapat mengakses dan memanipulasi isi dari class yang telah dibuat.

2.2 Inheritance

Inheritance adalah konsep dalam pemrograman berorientasi objek (OOP) di mana sebuah kelas (disebut subclass atau child class) dapat mewarisi atribut dan metode dari kelas lain (disebut superclass atau parent class).

2.3 Bahasa Pemrograman Python

Program aplikasi kasir ini dibangun menggunakan bahasa pemrograman Python. Python dipilih karena memiliki sintaks yang sederhana dan mudah dipahami, mendukung paradigma OOP secara penuh, serta banyak digunakan dalam pembelajaran dasar pemrograman maupun pengembangan aplikasi nyata. Dukungan Python terhadap konsep Class, Object, dan Inheritance memudahkan penerapan studi kasus aplikasi kasir ini.

2.4 Encapsulation

Encapsulation adalah proses membungkus data dan metode yang bekerja pada data tersebut menjadi satu unit dalam kelas, serta membatasi akses ke data secara langsung dari luar kelas. Hal ini membantu menjaga integritas data hanya kode tertentu yang boleh mengubah nilai-nilai penting dalam objek.

BAB III

ANALISIS DAN PERANCANGAN

3.1 Deskripsi Sistem

Aplikasi kasir sederhana ini dirancang untuk mencatat data produk yang dijual, mencatat detail belanja pelanggan, serta menghitung total harga, jumlah pembayaran, dan kembalian. Produk yang dijual dibedakan menjadi dua kategori, yaitu Makanan dan Minuman, yang masing-masing memiliki atribut tambahan yang berbeda meskipun tetap berasal dari Class induk yang sama, yaitu Produk.

3.2 Identifikasi Class

Berikut adalah Class-Class yang dibangun dalam program beserta atribut dan method utamanya.

Nama Class	Induk	Atribut Utama	Method Utama
Produk	-	id_produk, nama, harga, stok	init()
Makanan	Produk	kadaluarsa	init()
Minuman	Produk	ukuran	init()
Detail Belanja	-	produk, jumlah, subtotal	init()
Transaksi	-	detail_belanja, total, bayar, kembalian	cetak_struk()

3.3 Struktur Class

Class	Jenis Hubungan	Keterangan
Produk	Parent Class (Induk)	Menyimpan data dasar produk: id, nama, harga, stok
Makanan	Child Class dari Produk	Mewarisi Produk + atribut tambahan: kadaluarsa
Minuman	Child Class dari Produk	Mewarisi Produk + atribut tambahan: ukuran
DetailBelanja	Class Mandiri	Menyimpan 1 baris belanja (produk, jumlah, subtotal)
Transaksi	Class Mandiri	Mengelola seluruh proses pembayaran & cetak struk

3.4 Data Produk yang Tersedia

Id	Nama Produk	Jenis	Harga(Rp)	Stok	Atribut Khusus
1	Nasi Goreng Spesial	Makanan	25.000	20	Kadaluarsa: 2026-06-30
2	Mie Ayam Pangsit	Makanan	18.000	15	Kadaluarsa: 2026-05-30
3	Roti Bakar Cokelat	Makanan	15.000	10	Kadaluarsa: 2026-05-25
4	Es Teh Manis Jumbo	Minuman	5.000	50	Ukuran: Besar
5	Kopi Susu Gula Aren	Minuman	12.000	30	Ukuran: Sedang
6	Jus Alpukat	Minuman	15.000	12	Ukuran: Besar

3.5 Diagram Class

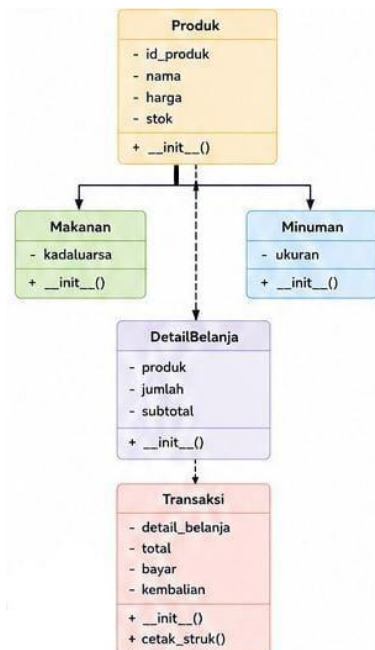


Diagram class di atas menggambarkan struktur lengkap program aplikasi kasir, yang terdiri dari lima class yaitu Produk, Makanan, Minuman, DetailBelanja, dan Transaksi, beserta dua jenis relasi yang menghubungkannya: inheritance (pewarisan) dan composition (komposisi/penggunaan).

1. Relasi Inheritance

Class Produk berperan sebagai *parent class* (superclass) yang menyimpan atribut dasar yang dimiliki seluruh produk, yaitu `id_produk`, `nama`, `harga`, dan `stok`, beserta satu method `__init__()` sebagai konstruktor.

Class Makanan dan Minuman merupakan *child class* (subclass) yang mewarisi seluruh atribut dari Produk. Garis solid dengan ujung segitiga kosong yang mengarah ke Produk menandakan relasi "is-a" artinya Makanan adalah sebuah Produk, dan Minuman adalah sebuah Produk. Masing-masing subclass menambahkan atributnya sendiri:

- Makanan menambahkan atribut `kadaluarsa`
- Minuman menambahkan atribut `ukuran`

2. Relasi Composition

Berbeda dengan Makanan dan Minuman, class DetailBelanja dan Transaksi tidak mewarisi class Produk. Keduanya berdiri sendiri sebagai class independen yang hanya menggunakan (uses) object dari class lain sebagai bagian dari atributnya relasi ini disebut composition, ditandai dengan garis putus-putus berlabel "uses".

- DetailBelanja memiliki atribut produk yang menyimpan satu object Produk (bisa berupa Makanan atau Minuman), jumlah, dan subtotal yang dihitung otomatis dari harga produk $\times$ jumlah.
- Transaksi memiliki atribut detail_belanja yang menyimpan kumpulan (list) object DetailBelanja, ditambah total, bayar, dan kembalian. Class ini juga memiliki method tambahan cetak_struk() untuk menampilkan struk pembelian.

3. Mengapa relasi ini dibedakan?

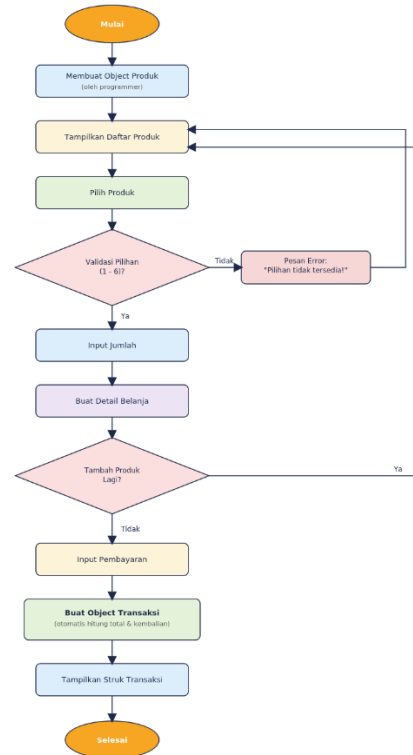
Pembedaan ini penting karena secara konsep keduanya berbeda makna:

- *Inheritance* digunakan ketika satu class benar-benar merupakan jenis khusus dari class lain (Makanan adalah jenis Produk).
- *Composition* digunakan ketika satu class hanya memanfaatkan/menyimpan object dari class lain sebagai bagian dari datanya, tanpa hubungan "jenis dari" (DetailBelanja bukan jenis dari Produk, ia hanya menyimpan referensi ke sebuah Produk).

Dengan struktur ini, program menjadi lebih modular: perubahan pada class Produk akan otomatis berlaku untuk Makanan dan Minuman tanpa perlu mengubah kode di kedua subclass tersebut, sementara DetailBelanja dan Transaksi tetap fleksibel karena hanya mengandalkan object yang diberikan kepadanya, tanpa terikat pada struktur pewarisan apa pun.

3.6 Flowchart Alur Program

5. Alur Program (Posisi Validasi Sudah Diperbaiki)



3.7 Alur Program

Berikut adalah alur logika program dari awal dimulai hingga struk pembelian ditampilkan:

1. Program menampilkan daftar 6 produk (3 Makanan dan 3 Minuman) beserta harganya.
2. Pengguna memilih nomor produk (1–6) yang ingin dibeli. Jika nomor di luar rentang 1–6, program menampilkan peringatan dan mengulang menu.
3. Pengguna memasukkan jumlah barang yang dibeli.
4. Program membuat object DetailBelanja baru dari produk dan jumlah yang dipilih, lalu memasukkannya ke dalam list detail_belanja, dan subtotal dihitung otomatis. Pengguna ditanya apakah ingin menambah produk lain. Jika menjawab "tidak", perulangan (loop) menu berhenti.
5. Pengguna memasukkan jumlah uang pembayaran.
6. Program membuat object Transaksi dari seluruh detail belanja dan jumlah uang yang dibayarkan, lalu otomatis menghitung total dan kembalian.
7. Program memanggil method cetak_struk() untuk menampilkan struk pembelian secara lengkap

BAB IV

IMPLEMENTASI DAN PENGUJIAN

Program aplikasi kasir ini diimplementasikan menggunakan bahasa pemrograman Python. Berikut adalah penjelasan implementasi setiap Class beserta potongan kode programnya.

4.1 Konsep OOP yang Diterapkan

Konsep OOP	Penerapan dalam Program
Class & Object	Produk, Makanan, Minuman, DetailBelanja, dan Transaksi adalah class; produk1 s.d. produk6 adalah object dari class tersebut.
Konstruktor (<code>__init__</code>)	Setiap class memiliki <code>__init__()</code> untuk menginisialisasi atribut saat object dibuat.
Inheritance	Makanan dan Minuman mewarisi class Produk menggunakan <code>super().__init__()</code> .
Encapsulation (sederhana)	Setiap class menyimpan data (atribut) dan perilaku (method) yang berkaitan dalam satu kesatuan, misalnya perhitungan subtotal di DetailBelanja dan total/kembalian di Transaksi.
Komposisi Object	Class DetailBelanja dan Transaksi memanfaatkan object dari class lain (Produk dan DetailBelanja) sebagai bagian dari atributnya.

4.2 Class Produk (Parent Class)

Class Produk berperan sebagai parent class (class induk) yang menyimpan atribut paling dasar yang dimiliki oleh seluruh jenis produk, yaitu `id_produk`, `nama`, `harga`, dan `stok`. Atribut-atribut ini diinisialisasi melalui konstruktor `__init__( )` yang akan dipanggil setiap kali sebuah object Produk dibuat.

```
# Class Produk (Parent Class)
class Produk:

    # Konstruktor untuk menyimpan data produk
    def __init__(self, id_produk, nama, harga, stok):
        self.id_produk = id_produk
        self.nama = nama
        self.harga = harga
        self.stok = stok
```

4.3 Class Makanan (Inheritance dari Produk)

Class Makanan merupakan child class yang mewarisi seluruh atribut dari class Produk menggunakan fungsi `super().__init__()`. Pewarisan ini membuat class Makanan tidak perlu menuliskan ulang kode untuk `id_produk`, `nama`, `harga`, dan `stok`, melainkan cukup memanggil konstruktor parent-nya. Class ini menambahkan satu atribut khusus, yaitu `kadaluarsa`, yang merepresentasikan tanggal kadaluarsa produk makanan.

```
# Class Makanan mewarisi class Produk
class Makanan(Produk):

    # Menambahkan atribut tanggal kadaluarsa
    def __init__(self, id_produk, nama, harga, stok, kadaluarsa):
        super().__init__(id_produk, nama, harga, stok)
        self.kadaluarsa = kadaluarsa
```

4.4 Class Minuman (Inheritance dari Produk)

Sama seperti class Makanan, class Minuman juga mewarisi class Produk. Perbedaannya, class ini menambahkan atribut `ukuran` (misalnya "Besar" atau "Sedang") yang relevan untuk produk minuman.

```
# Class Minuman mewarisi class Produk
class Minuman(Produk):

    # Menambahkan atribut ukuran
    def __init__(self, id_produk, nama, harga, stok, ukuran):
        super().__init__(id_produk, nama, harga, stok)
        self.ukuran = ukuran
```

4.5 Class DetailBelanja

Class DetailBelanja digunakan untuk merepresentasikan satu baris transaksi belanja, yaitu produk apa yang dibeli dan berapa jumlahnya. Class ini menyimpan object produk (bisa berupa object Makanan ataupun Minuman), jumlah pembelian, serta secara

otomatis menghitung subtotal dengan mengalikan harga produk dengan jumlah barang yang dibeli.

```
# Class DetailBelanja untuk menyimpan detail pembelian
class DetailBelanja:

    def __init__(self, produk, jumlah):
        self.produk = produk
        self.jumlah = jumlah
        self.subtotal = produk.harga * jumlah
```

4.6 Class Transaksi

Class Transaksi bertugas mengelola keseluruhan proses pembayaran. Class ini menerima daftar object DetailBelanja serta jumlah uang yang dibayarkan oleh pembeli. Di dalam konstruktornya, program melakukan perulangan (for) untuk menjumlahkan seluruh subtotal menjadi total belanja, kemudian menghitung kembalian dengan mengurangi total dari uang yang dibayarkan.

```
# Class Transaksi untuk menghitung total pembayaran
class Transaksi:

    def __init__(self, detail_belanja, bayar):

        self.detail_belanja = detail_belanja
        self.total = 0

        # Menghitung total harga
        for item in detail_belanja:
            self.total += item.subtotal

        self.bayar = bayar
        self.kembalian = bayar - self.total

    # Menampilkan struk pembayaran
    def cetak_struk(self):

        print("\nSTRUK PEMBELIAN")

        for item in self.detail_belanja:
            print("Produk   :", item.produk.nama)
            print("Harga    : Rp", item.produk.harga)
            print("Jumlah   :", item.jumlah)
            print("Subtotal : Rp", item.subtotal)
            print("-" * 30)

        print("Total      : Rp", self.total)
        print("Bayar     : Rp", self.bayar)
        print("Kembalian  : Rp", self.kembalian)
```

Method cetak_struk() berfungsi untuk menampilkan struk pembayaran ke layar, mencakup rincian setiap produk yang dibeli, total belanja, jumlah uang yang dibayarkan, dan kembalian yang diterima pembeli.

4.7 Pembuatan Object dan Eksekusi Program

Tahap pertama adalah membuat object dari class Makanan dan Minuman yang telah didefinisikan sebelumnya. Setiap baris kode berikut akan memanggil konstruktor `__init__( )` milik class Makanan atau Minuman, sehingga setiap object langsung memiliki nilai `id_produk`, `nama`, `harga`, `stok`, serta atribut khususnya (`kadaluarsa` untuk Makanan, `ukuran` untuk Minuman).

```
# Object Makanan
produk1 = Makanan(1, "Nasi Goreng Spesial", 25000, 20, "2026-06-30")
produk2 = Makanan(2, "Mie Ayam Pangsit", 18000, 15, "2026-05-30")
produk3 = Makanan(3, "Roti Bakar Cokelat", 15000, 10, "2026-05-25")

# Object Minuman
produk4 = Minuman(4, "Es Teh Manis Jumbo", 5000, 50, "Besar")
produk5 = Minuman(5, "Kopi Susu Gula Aren", 12000, 30, "Sedang")
produk6 = Minuman(6, "Jus Alpukat", 15000, 12, "Besar")
```

Sebagai contoh, pada baris `produk1 = Makanan(1, "Nasi Goreng Spesial", 25000, 20, "2026-06-30")`, Python akan menjalankan konstruktor class Makanan dengan nilai `id_produk=1`, `nama="Nasi Goreng Spesial"`, `harga=25000`, `stok=20`, dan `kadaluarsa="2026-06-30"`. Di dalam konstruktor tersebut, `super().__init__( )` akan dipanggil terlebih dahulu untuk mengisi atribut `id_produk`, `nama`, `harga`, dan `stok` melalui class Produk, kemudian atribut `kadaluarsa` diisi secara khusus oleh class Makanan. Hal yang sama berlaku untuk `produk4` hingga `produk6` yang merupakan object dari class Minuman.

4.8 Menyimpan Seluruh Produk ke dalam List

Keenam object produk yang telah dibuat kemudian dikumpulkan ke dalam satu variabel list bernama `daftar_produk`. Tujuannya agar seluruh produk dapat diakses dan ditampilkan secara berurutan menggunakan perulangan (loop), tanpa harus menulis nama setiap object satu per satu di banyak tempat.

```
# Menyimpan semua produk ke dalam list
daftar_produk = [
    produk1,
    produk2,
    produk3,
    produk4,
    produk5,
    produk6
]

# Menyimpan detail belanja
detail_belanja = []
```

Selain itu, dibuat juga sebuah list kosong bernama `detail_belanja`. List ini nantinya akan menampung object-object `DetailBelanja`, yaitu setiap produk yang berhasil dipilih dan dibeli oleh pengguna selama program berjalan.

4.9 Menu Pembelian (Perulangan while)

Inti dari program utama berada pada struktur perulangan `while True`;, yang akan terus berjalan berulang-ulang selama pengguna masih ingin membeli produk. Perulangan ini hanya akan berhenti apabila pengguna menjawab "tidak" pada pertanyaan "Tambah produk lagi?".

```
# Menu pembelian
while True:

    print("\nDAFTAR PRODUK")

    # Menampilkan daftar produk
    for produk in daftar_produk:
        print(f"{produk.id_produk}. {produk.nama} - Rp {produk.harga}")

    # Memilih produk
    pilihan = int(input("\nPilih Produk (1-6): "))

    # Validasi pilihan
    if pilihan < 1 or pilihan > 6:
        print("Pilihan tidak tersedia!")
        continue

    # Memasukkan jumlah pembelian
    jumlah = int(input("Jumlah Item : "))

    # Membuat object DetailBelanja
    produk = daftar_produk[pilihan - 1]
    detail_belanja.append(DetailBelanja(produk, jumlah))

    # Konfirmasi menambah produk
    lagi = input("Tambah produk lagi? (iya/tidak): ").lower()

    if lagi == "tidak":
        break
```

Penjelasan langkah demi langkah pada blok kode di atas adalah sebagai berikut:

- 1) Menampilkan daftar produk perulangan `for` digunakan untuk membaca satu per satu object pada `daftar_produk`, lalu mengambil atribut `id_produk`, `nama`, dan `harga` masing-masing untuk ditampilkan ke pengguna.
- 2) Input pilihan produk fungsi `input()` membaca angka 1-6 yang dimasukkan pengguna dan diubah menjadi tipe data integer menggunakan `int()`.
- 3) Validasi pilihan jika angka yang dimasukkan kurang dari 1 atau lebih dari 6, program menampilkan pesan "Pilihan tidak tersedia!" lalu perintah `continue` akan langsung mengulang perulangan `while` dari awal tanpa menjalankan baris kode di bawahnya.

- 4) Input jumlah barang pengguna memasukkan jumlah item yang ingin dibeli untuk produk yang telah dipilih.
- 5) Mengambil object produk `daftar_produk[pilihan - 1]` digunakan untuk mengambil object produk sesuai indeks list (dikurangi 1 karena indeks list pada Python dimulai dari 0, sedangkan pilihan menu dimulai dari 1).
- 6) Membuat object `DetailBelanja` `DetailBelanja(produk, jumlah)` membuat object baru yang otomatis menghitung subtotal, kemudian object tersebut langsung ditambahkan (`append`) ke dalam list `detail_belanja`.
- 7) Konfirmasi tambah produk pengguna ditanya apakah ingin membeli produk lain. Jika jawabannya "tidak", perintah `break` akan menghentikan perulangan `while` secara permanen.

5.0 Proses Pembayaran dan Pembuatan Object Transaksi

Setelah pengguna selesai memilih seluruh produk yang ingin dibeli (keluar dari perulangan `while`), program akan meminta pengguna memasukkan jumlah uang pembayaran. Nilai ini kemudian digunakan untuk membuat sebuah object baru dari class `Transaksi`.

```
# Proses pembayaran
bayar = int(input("\nMasukkan Uang Pembayaran : Rp "))

# Membuat object Transaksi
transaksi = Transaksi(detail_belanja, bayar)

# Menampilkan struk pembayaran
transaksi.cetak_struk()
```

Pada baris `transaksi = Transaksi(detail_belanja, bayar)`, seluruh isi list `detail_belanja` (yang berisi satu atau lebih object `DetailBelanja` hasil dari menu pembelian sebelumnya) dikirim sebagai parameter ke konstruktor class `Transaksi`, bersama dengan jumlah uang yang dibayarkan. Di dalam konstruktor tersebut, program otomatis menjumlahkan seluruh subtotal menjadi total belanja dan menghitung kembalian. Terakhir, method `transaksi.cetak_struk()` dipanggil untuk menampilkan rincian struk pembelian secara lengkap ke layar, menandai berakhirnya eksekusi program.

5.1 Hasil Output Program

Berikut adalah hasil keluaran (output) program setelah dijalankan menggunakan interpreter Python.

```
***
DAFTAR PRODUK
1. Nasi Goreng Spesial - Rp 25000
2. Mie Ayam Pangsit - Rp 18000
3. Roti Bakar Cokelat - Rp 15000
4. Es Teh Manis Jumbo - Rp 5000
5. Kopi Susu Gula Aren - Rp 12000
6. Jus Alpukat - Rp 15000

Pilih Produk (1-6): 1
Jumlah Item : 2
Tambah produk lagi? (iya/tidak): iya

DAFTAR PRODUK
1. Nasi Goreng Spesial - Rp 25000
2. Mie Ayam Pangsit - Rp 18000
3. Roti Bakar Cokelat - Rp 15000
4. Es Teh Manis Jumbo - Rp 5000
5. Kopi Susu Gula Aren - Rp 12000
6. Jus Alpukat - Rp 15000

Pilih Produk (1-6): 6
Jumlah Item : 1
Tambah produk lagi? (iya/tidak): tidak

Masukkan Uang Pembayaran : Rp 100000

STRUK PEMBELIAN
Produk   : Nasi Goreng Spesial
Harga    : Rp 25000
Jumlah   : 2
Subtotal : Rp 50000
-----
Produk   : Jus Alpukat
Harga    : Rp 15000
Jumlah   : 1
Subtotal : Rp 15000
-----
Total      : Rp 65000
Bayar      : Rp 100000
Kembalian  : Rp 35000
```

5.1 Tabel Hasil Pengujian

No.	Skenario Pengujian	Input	Output yang Diharapkan	Status
1.	Memilih produk valid (makanan)	Pilih: 1, Jumlah: 2	Produk "Nasi Goreng Spesial" masuk ke detail belanja, subtotal Rp 50.000	Berhasil
2.	Menambah lebih dari satu produk berbeda	Pilih: 4, Jumlah: 1, lalu "tidak"	Produk "Es Teh Manis Jumbo" ditambahkan, total belanja menjadi Rp 55.000	Berhasil
3.	Memilih produk dengan nomor tidak tersedia	Pilih: 7	Muncul pesan "Pilihan tidak tersedia!" dan kembali ke daftar produk	Berhasil
4.	Pembayaran kurang dari total	Total: Rp 55.000, Bayar: Rp 50.000	Kembalian bernilai negatif, Rp -5.000	Berhasil (sesuai logika, tapi lihat catatan)
5.	Pembayaran pas sesuai total	Total: Rp 12.000, Bayar: Rp 12.000	Kembalian Rp 0	Berhasil
6.	Pembelian produk dengan jumlah lebih dari 1 (minuman)	Pilih: 6, Jumlah: 3	Subtotal "Jus Alpukat" = Rp 45.000, kembalian Rp 5.000 dari bayar Rp 50.000	Berhasil
7.	Mencetak struk pembelian setelah transaksi selesai	Setelah input pembayaran	Struk tampil lengkap dengan daftar produk, subtotal, total, bayar, dan kembalian	Berhasil

BAB V

KESIMPULAN

Berdasarkan hasil perancangan, implementasi, dan pengujian Aplikasi Kasir Sederhana yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut.

Pertama, konsep Class dan Object berhasil diterapkan dengan baik dalam membangun sistem aplikasi kasir ini. Class Produk, DetailBelanja, dan Transaksi masing-masing dirancang sebagai struktur dasar (blueprint) yang mendefinisikan atribut dan method, kemudian diinstansiasi menjadi Object-Object konkret yang merepresentasikan data produk, rincian belanja, dan transaksi yang sesungguhnya. Hal ini membuktikan bahwa Class berperan sebagai rancangan, sedangkan Object adalah hasil nyata dari rancangan tersebut, sebagaimana dijelaskan pada landasan teori.

Kedua, konsep Inheritance terbukti efektif diterapkan pada Class Produk sebagai superclass yang diturunkan menjadi Class Makanan dan Class Minuman sebagai subclass. Melalui penggunaan `super().__init__()`, kedua subclass tersebut dapat mewarisi atribut umum (`id_produk`, `nama`, `harga`, `stok`) tanpa perlu menuliskan ulang kode, sekaligus menambahkan atribut khusus yang membedakan keduanya, yaitu kadaluarsa pada Makanan dan ukuran pada Minuman. Pendekatan ini menunjukkan manfaat utama Inheritance dalam mengurangi duplikasi kode dan menjaga struktur program tetap modular.

Ketiga, integrasi antar Class (Produk, DetailBelanja, dan Transaksi) memungkinkan sistem menghitung subtotal, total harga, serta kembalian secara otomatis dan akurat, sehingga mengatasi permasalahan yang disebutkan dalam latar belakang, yaitu kesalahan perhitungan manual dan lambatnya proses transaksi.

Dengan demikian, penerapan paradigma OOP, khususnya konsep Class, Object, dan Inheritance, pada aplikasi kasir sederhana ini berhasil menjawab rumusan masalah serta mencapai tujuan penelitian, yaitu menghasilkan program yang modular, mudah dikembangkan, dan dapat mengotomatisasi proses transaksi secara cepat dan tepat. Ke depannya, sistem ini dapat dikembangkan lebih lanjut dengan menambahkan konsep Encapsulation yang lebih ketat (misalnya melalui atribut `private`) maupun Polymorphism pada method-method yang dimiliki masing-masing subclass.

DAFTAR PUSTAKA

Jubilee Enterprise. (n.d.). *Profil penulis*. Elex Media Komputindo.

KMTech. (n.d.). *Pemrograman berorientasi objek: Konsep dasar dan implementasinya dalam bahasa pemrograman*.

Masuk PTN. (n.d.). *Inheritance, encapsulation, polymorphism*.

Nabila Kartika Putri. (n.d.). *Inheritance (pewarisan) pada OOP*. Medium.

Suryo Mujahid. (2020, August 20). *Mengenal class, object, attribute dan method dalam OOP*.

Wibawa, A. P. (n.d.). *Google Scholar profile*. Google Scholar.